

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Дьячкова Елизавета

J3212

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задачей данной лабораторной работы является интеграция ранее сверстанного веб-приложения из Лабораторной №1 с внешним API, основными требованиями является настройка мокового API с подключением JSON Server для создания имитационного API, реализовать авторизацию, сделать переход на асинхронное взаимодействие, а также обеспечить полную функциональность приложения, то есть:

1. Регистрация и вход пользователя с сохранением сессии.
2. Поиск и фильтрация мероприятий с получением данных с сервера.
3. Просмотр детальной информации о мероприятии (афиша, место проведения, схема зала, отзывы).
4. Покупка билета
5. Личный кабинет пользователя с отображением купленных билетов и возможностью выхода из аккаунта
6. Личный кабинет организатора с возможностью создания новых мероприятий и управления созданными событиями.

Ход работы

1. Настройка мокового API

Для имитации работы серверной части установила и настроила пакеты `json-server` и `json-server-auth`, это я сделала для полноценного мокового API с поддержкой авторизации. В файле базы данных `db.json` были определены три основные коллекции:

`users` - хранит зарегистрированных пользователей. Изначально коллекция пуста и заполняется по мере регистрации новых пользователей.

`events` - содержит список мероприятий. Для каждого события предусмотрены поля: название, тип (концерт, театр, фестиваль), город, дата, место проведения, цена, описание, отзывы, а также идентификатор организатора, что позволяет связывать событие с конкретным пользователем-организатором.

`tickets` - хранит информацию о купленных билетах. Каждая запись связывает пользователя и мероприятие, а также дублирует название, дату и цену для удобства отображения в личном кабинете.

Также в файле `server.js` настроен `json-server` с подключением `json-server-auth` для поддержки JWT-авторизации, для удобства запуска в `package.json` был добавлен скрипт `"api": "node server.js"`. Сервер запускается на локальном порту 3001, а также становится доступен по адресу.

2. Реализация взаимодействия с API во фронтенде

Вся логика работы с данными переведена на асинхронные запросы с использованием `fetch`. А мой основной код взаимодействия находится в файле `script.js`.

Про авторизацию:

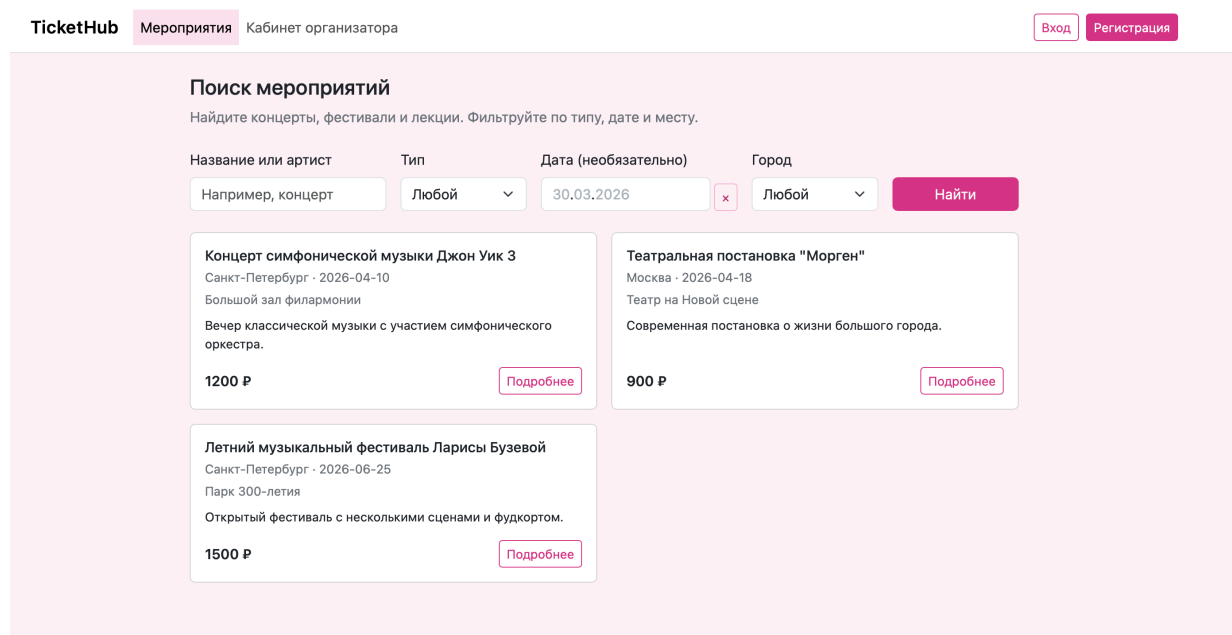
После успешного входа или регистрации токен сохраняется в `localStorage` и автоматически подставляется в заголовок `Authorization: Bearer` для всех последующих запросов. Навигация динамически обновляется, то есть кнопки «Вход/Регистрация» скрываются, появляется «Личный кабинет».

Про регистрацию и вход (страницы `register.html` и `login.html`):

Данные отправляются на эндпоинты `/register` и `/login`. При успешном ответе токен сохраняется, пользователь перенаправляется в личный кабинет.

Про главную страница с поиском (`index.html`):

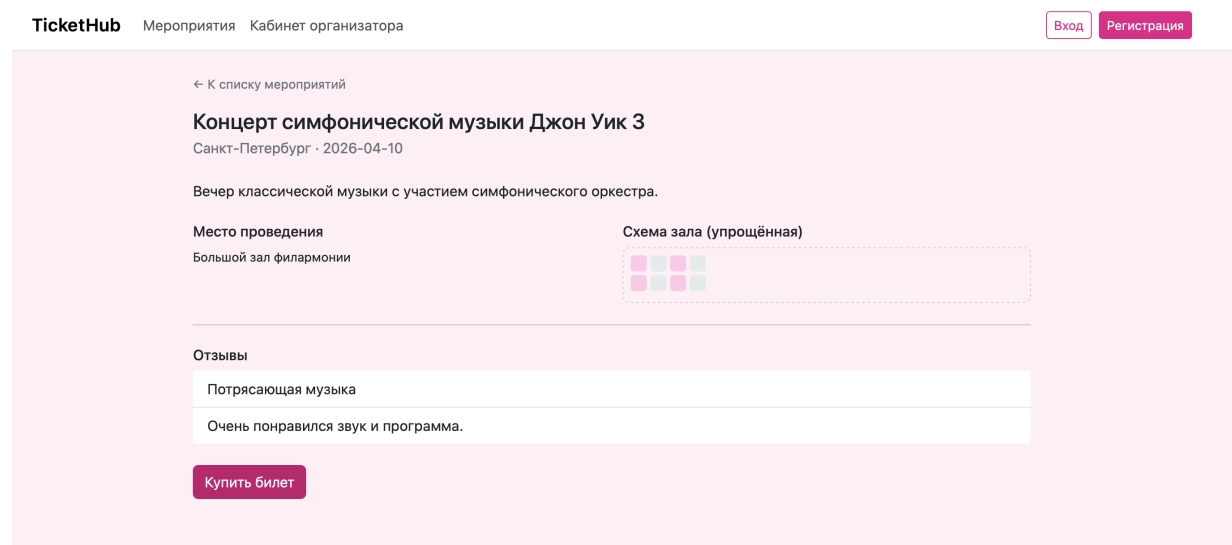
При загрузке выполняется запрос `GET /events` для получения списка мероприятий. Данные кэшируются в `sessionStorage` для ускорения переходов. Реализована фильтрация по названию, типу, дате и городу. Результаты отображаются в виде карточек.



Про страницу мероприятия (`event.html`):

`id` мероприятия извлекается из URL-параметра `?id=...`. Выполняется запрос `GET /events/{id}`. Если прямой запрос не находит мероприятие, срабатывает `fallback`: загружается полный список событий и поиск в кэше `sessionStorage`. После получения данных заполняются заголовок, описание, место проведения, схема зала и отзывы. При клике на кнопку «Купить билет» проверяется авторизация. Если пользователь вошел,

отправляется POST запрос на /tickets с данными билета. Если нет, то перенаправление на страницу входа.



Личный кабинет пользователя (profile.html):

При загрузке страницы проверяется наличие токена и данных о пользователе. Если их нет - редирект на страницу входа. Выполняется запрос GET /tickets?userId={id} для получения билетов текущего пользователя. Список билетов динамически отрисовывается, при отсутствии билетов выводится сообщение. Кнопка «Выйти» очищает localStorage и перенаправляет на главную.

Личный кабинет организатора (organizer.html):

Аналогично проверяется авторизация. Загружаются все мероприятия через GET /events, затем фильтруются те, у которых поле organizerUserId совпадает с id текущего пользователя, так отображается список созданных им событий.

Форма создания события отправляет POST запрос на /events с данными нового мероприятия, подставляя organizerUserId. После успешного создания список событий организатора обновляется без перезагрузки страницы.

TicketHub

Мероприятия

Кабинет организатора

Вход

Регистрация

Кабинет организатора

Создать событие

Название

Город

Выберите город

Дата

30.03.2026

Тип

Концерт

Цена от

Добавить событие

Мои события

Войдите, чтобы управлять событиями.

3. Реализация UI/UX улучшений

Toasts - для обратной связи с пользователем используется компонент Bootstrap Toast. В коде реализована функция `showToast (message)`, которая вызывается при успешном входе, покупке билета или возникновении ошибки.

Обработка ошибок - все запросы к API обернуты в `.catch()`. При ошибках сети, недоступности сервера или некорректных данных пользователю выводится информативное сообщение через `showToast` или через элемент `apiStatus` на странице.

Состояние загрузки - на страницах с динамическими данными используются атрибуты `aria-busy="true"` и их программное снятие после завершения загрузки, что улучшает пользовательский опыт.

Адаптивность - для верстки использованы классы Bootstrap (`row`, `col-12`, `col-md-6`, `col-lg-4`), что обеспечивает корректное отображение интерфейса на различных устройствах от мобильных телефонов до настольных компьютеров

Вывод

В ходе работы я научилась подключать моковый API с помощью `json-server` и реализовала полноценную авторизацию. Все данные теперь загружаются с сервера через `fetch`, а не хранятся локально. Приложение поддерживает регистрацию, вход, поиск и фильтрацию мероприятий, покупку билетов, личный кабинет пользователя и кабинет организатора с созданием событий.